

- 1 What this document is
- 2 Step 0: from the loggers' export tables to one record (00a_compile_logger_readings.R)
- 3 Step 1: from logger readings to sensor-nights (00_prepare_inputs.R)
- 4 Step 2: describing each site (01_site_predictors.R)
- 5 Step 3: the mixed-effects models (02_models.R)
- 6 Step 4: the checks (06_elevation_check.R, 07_sensitivity_checks.R)
- 7 Step 5: figures and tables (03_figures.R, 05_thesis_tables.R, 08_appendix_figures.R)
- 8 Reproducing everything
- 9 Verification: an independent replication in Python
- 10 References for the software

Island Urban Climatology: the analysis, step by step

From the logger readings to every number in the thesis: each processing rule, each model, each check, and the package that does it

Justin Kellum, Department of Geography and Environment, University of Hawai'i at Mānoa

27 September 2026

1 What this document is

The thesis *Island Urban Climatology: Measuring and Modeling Nocturnal Heat in Leeward Coastal O'ahu, Hawai'i* asks which properties of the urban surface explain where night air is warmer or cooler than the rest of a district, using 74 logger sites in Honolulu and 'Ewa on calm, clear nights. Every number, table and figure in it is produced from the files in `data/raw/` by the scripts in `analysis/`, in this order:

Script	What it does	Runtime
00a_compile_logger_readings.R	reads the loggers' export tables, separates the download application's metadata rows from the readings, writes one record of readings and one table of logger metadata, and checks the stored record against the exports	seconds
00_prepare_inputs.R	reads the logger readings, snaps the loggers' clocks to 30-minute bins, computes the district network median, joins the hourly wind and cloud series, classifies every night, and writes the sensor-night table that all models use	seconds

Script	What it does	Runtime
01_site_predictors.R	computes the eight urban-form descriptors of every site at 50, 100 and 200 m from the C-CAP 1 m land-cover masks, the FEMA building footprints (including a ray-cast sky view factor) and the GSHHG shoreline	~7 min
02_models.R	fits every mixed-effects model of the thesis (scale selection, null models, single descriptors, full and district models, best subsets, interactions, night effect, threshold sensitivity, cross-validation, site effects, shared nights) and writes results/thesis_results.json and the CSV tables	~1 min
03_figures.R	draws 19 of the 22 main-text figures (all but the two photographs and the terrain figure) with ggplot2 and sf/terra	~10 min
04_replication_check.R	compares the R results with the independent Python implementation in replication/ (Section 9)	seconds
05_thesis_tables.R	turns the results into 21 of the thesis's 22 tables (all but the descriptor definitions, which are text), in Markdown, exactly as they appear in the document	seconds
06_elevation_check.R	the terrain check (thesis Section 4.8): adds the USGS 3DEP elevation of each site to the models, re-runs the best-subset search and the cross-validation with it, and draws Figure 19	~1 min
07_sensitivity_checks.R	three site-level checks (thesis Sections 4.8 and 5.4): the models without the warmest site, with a different height for the buildings whose height the inventory does not record, and with the sky view factor of positions inside footprints recomputed	~7 min
08_appendix_figures.R	the land-cover and terrain maps and the sky view factor diagram of Appendix B	~1 min

`run_all.R` runs them in order (about 30 minutes). This document walks through the chain in the same order, shows the code that does each step, runs the fast parts live, and states which package does what and why it was chosen. It is written so that a reader who knows R but has not seen the project can follow, and rerun, every decision.

Hide

```
source(file.path(root, "analysis", "helpers.R"), encoding = "UTF-8")          # paths, constants,
small_utilities
source(file.path(root, "analysis", "00_prepare_inputs.R"), encoding = "UTF-8") # defines
load_binned(), night_inventory(), sensor_nights()
suppressPackageStartupMessages({ library(lme4); library(lmerTest); library(performance); library(car);
library(sf); library(terra); library(ggplot2); library(kableExtra) })
```

1.1 The packages, and what each one does here

Package	Role	Functions used	Reference
base R 4.3	the language; all arithmetic, <code>median()</code> , <code>cor()</code> , <code>lm()</code> for the stepwise check, <code>wilcox.test()</code> , <code>cor.test()</code>		R Core Team (2024)
dplyr, tidyr, tibble, purrr	grouped summaries, joins and reshaping of the record (one row per logger and bin → one row per sensor-night)	<code>group_by()</code> , <code>summarise()</code> , <code>inner_join()</code> , <code>pivot_wider()</code>	Wickham et al. (2019)
readr	fast, typed CSV input and output	<code>read_csv()</code> , <code>write_csv()</code>	Wickham et al. (2019)
lubridate	rounding of the loggers' time stamps to the nearest 30 minutes	<code>round_date()</code> , <code>ymd_hms()</code>	Grolemund & Wickham (2011)
jsonlite	the results file that the tables and figures read	<code>write_json()</code> , <code>fromJSON()</code>	Ooms (2014)
sf	vector geometry: projecting the sites, buffering, intersecting footprints with circles, ray casting, distance to the shoreline	<code>st_transform()</code> , <code>st_buffer()</code> , <code>st_intersects()</code> , <code>st_intersection()</code> , <code>st_distance()</code>	Pebesma (2018)
terra	raster access: the 1 m C-CAP masks as virtual mosaics, pixel extraction inside circles, aggregation for the base maps	<code>vrt()</code> , <code>extract()</code> , <code>crop()</code> , <code>aggregate()</code>	Hijmans (2024)
lme4	the mixed-effects models: random intercept per site, REML and ML fits	<code>lmer()</code> , <code>fixef()</code> , <code>ranef()</code> , <code>VarCorr()</code> , <code>logLik()</code> , <code>AIC()</code> , <code>predict()</code>	Bates et al. (2015)
lmerTest	Satterthwaite degrees of freedom and t tests for the coefficients	<code>lmer()</code> (wrapper), <code>summary()</code>	Kuznetsova et al. (2017)
performance	marginal and conditional R^2 (Nakagawa) and ICC, as a check on the manual formulas	<code>r2_nakagawa()</code> , <code>icc()</code>	Lüdtke et al. (2021)
car	variance inflation factors of the predictors	<code>vif()</code>	Fox & Weisberg (2019)
ggplot2, patchwork, scales, cowplot	all figures; multi-panel layout; colour scales; legend extraction	<code>ggplot()</code> , <code>geom_sf()</code> , <code>geom_raster()</code> , <code>wrap_plots()</code>	Wickham (2016); Pedersen (2024)
rmarkdown, knitr, kableExtra	this document	<code>render()</code> , <code>kable()</code>	Xie et al. (2018)

Why these and not others: `lme4` is the reference implementation of linear mixed models in R; `lmerTest` adds the Satterthwaite approximation that `lme4` deliberately omits, so that each coefficient has a defensible p-value; `sf` and `terra` are the current standard vector and raster packages (the older `sp` / `rgdal` / `raster`

stack was retired in 2023); performance is used only to confirm that the R^2 and ICC computed by hand from `VarCorr()` are right. The whole chain is base R plus these packages, with no compiled code of our own and nothing hidden. `results/sessionInfo.txt` records the exact versions of the run that produced the thesis.

2 Step 0: from the loggers' export tables to one record (`00a_compile_logger_readings.R`)

The measurement exists first as the tables produced by the manufacturer's application when the recovered loggers were read out: one table per district in `data/raw/exports/`, one row per reading (logger, date, time, °C) and one row per logger carrying the application's metadata as JSON (model, serial number, sample rate, mission start, sample counts). Nothing in these tables has been edited. `00a_compile_logger_readings.R` separates the metadata rows from the readings, writes `data/raw/logger_readings.csv` (every reading, sorted by district, logger and time) and `data/raw/logger_metadata.csv` (one row per logger), and refuses to proceed if a reading is duplicated or if the stored record disagrees with the exports in any reading:

Hide

```
md <- read_csv(file.path(RAW, "logger_metadata.csv"), show_col_types = FALSE)
md %>% count(region, model, sample_rate_s, resolution_c, app_version) %>% tab(caption = "The loggers,
as the download application describes them")
```

The loggers, as the download application describes them

region	model	sample_rate_s	resolution_c	app_version	n
Ewa	DS1921H	1800	0.5	3.5.4	38
Honolulu	DS1921H	1800	0.5	3.4.14	33
Honolulu	DS1921H	1800	0.5	3.5.4	3

Hide

```
md %>% group_by(region) %>% summarise(loggers = n(), readings = sum(readings), first = min(first), last
= max(last),
                                   rollover = sum(rollover)) %>% tab()
```

region	loggers	readings	first	last	rollover
Ewa	38	77824	2024-11-14 20:31:00	2024-12-27 14:32:00	0
Honolulu	36	73728	2024-11-23 21:30:00	2025-01-20 08:01:00	2

Every logger returned exactly 2,048 readings, the capacity of the DS1921H memory (42.7 days at 30 minutes): for 72 loggers the mission simply stopped recording when the memory was full, and for two (HNL01 and HNL34) the memory rolled over, so their record is the last 42.7 days before read-out rather than the first. This is why the Honolulu network thins out after early January (Figure 5 of the thesis), and it is a property of the instrument, not a choice made in processing.

3 Step 1: from logger readings to sensor-nights (`00_prepare_inputs.R`)

3.1 What comes in

Three files in `data/raw/` and `data/external/`. The logger readings are the record compiled in Step 0: one row per logger and time stamp (no reading has been altered or removed). The sites file gives each logger's district, position and the setting recorded at deployment. The weather series gives the hourly 10 m wind speed and total cloud cover at the grid point nearest each district (21.34°N 157.80°W for Honolulu, 21.34°N 158.07°W for Ewa), retrieved from the Open-Meteo historical-weather API for the whole deployment period by `analysis/inputs/fetch_weather_series.py` (rerunning it reproduces the committed file byte for byte). Open-Meteo's default historical dataset combines ERA5, ERA5-Land and the ECMWF IFS high-resolution analysis; for this period and place the values are those of the IFS analysis (9 km), which we verified value by value against each model of the API. The thesis calls it the ECMWF analysis.

[Hide](#)

```
hh_raw <- read_csv(file.path(RAW, "logger_readings.csv"), col_types = cols(datetime = col_character(),  
.default = col_guess()))  
sites <- read_csv(file.path(RAW, "sites.csv"), show_col_types = FALSE)  
wx_raw <- read_csv(file.path(EXT, "weather_hourly_series.csv"), col_types = cols(datetime =  
col_character(), .default = col_guess()))  
cat(nrow(hh_raw), "readings from", n_distinct(hh_raw$sensor_id), "loggers;", nrow(sites), "sites;",  
nrow(wx_raw), "weather rows\n")
```

```
#> 151552 readings from 74 loggers; 74 sites; 10960 weather rows
```

[Hide](#)

```
head(hh_raw, 3)
```

region <chr>	sensor_id <chr>	datetime <chr>	temp_c <dbl>
Honolulu	HNL01	2024-12-08 16:31:00	27.375
Honolulu	HNL01	2024-12-08 17:01:00	26.125
Honolulu	HNL01	2024-12-08 17:31:00	25.000

3 rows

[Hide](#)

```
sites %>% count(region, setting) %>% tab(caption = "The logger sites by district and recorded setting")
```

The logger sites by district and recorded setting

region	setting	n
Ewa	Dense Urban	22
Ewa	Grass Area with Tree Border	3
Ewa	High Canopy Park	1
Ewa	Managed Grassland	4
Ewa	Multi-use Recreational Park	2
Ewa	Unmanaged Grassland	6
Honolulu	Dense Urban	15
Honolulu	Grass Area with Tree Border	8

region	setting	n
Honolulu	High Canopy Park	4
Honolulu	Managed Grassland	3
Honolulu	Managed Wetland	1
Honolulu	Multi-use Recreational Park	4
Honolulu	Unmanaged Grassland	1

Every logger reports every 30 minutes, but the loggers were started by hand, so their clocks differ by up to two minutes:

Hide

```
table(minute(read_stamps(hh_raw$datetime)))
```

```
#>
#>      0      1      2    30    31    32
#> 1024 65536 9216 1024 65536 9216
```

This matters for the reference. A median taken over the loggers that share an exact minute stamp would compare the loggers on a :02/:32 cadence (five in Honolulu, four in 'Ewa) with four or five loggers instead of the district network, and one logger (HNL47, alone on :00/:30) with itself. The first processing rule removes the problem.

3.2 Rule 1: snap every reading to the nearest 30-minute bin

Hide

```
hh <- hh_raw %>%
  mutate(datetime = read_stamps(datetime),          # parsed as clock time, no time-zone
  shift
    time_bin = round_date(datetime, "30 minutes"),  # lubridate: :01 -> :00, :31 -> :30,
:02 -> :00 ...
    hour_dec = hour(time_bin) + minute(time_bin) / 60,
    night_date = format(time_bin - hours(NIGHT_END), "%Y-%m-%d"), # a diurnal date runs 06:00 ->
06:00
    is_night = hour_dec >= NIGHT_START | hour_dec < NIGHT_END) # 18:00 -> 06:00 HST
```

`round_date()` puts all loggers of a district on the same 30-minute grid; because no stamp ever falls exactly on the quarter-hour, rounding is unambiguous. A diurnal date starts at 06:00 so that a whole night (18:00–06:00) belongs to one date.

3.3 Rule 2: the district network median, and ΔT

Hide

```
med <- hh %>% group_by(region, time_bin) %>%
  summarise(network_median_temp = median(temp_c), n_sensors_active = n_distinct(sensor_id), .groups =
"drop")
hh <- hh %>% inner_join(med, by = c("region", "time_bin")) %>% mutate(dT_network = temp_c -
network_median_temp)
```

The response of the whole thesis is `dT_network`: a logger's air temperature minus the median of all loggers of its district in the same bin. The median (not the mean) is used because it is insensitive to a few extreme sites; because every district is centred on its own median, the two districts can only be compared in absolute terms

on the nights they share (Section 4.5 below; thesis Section 4.10).

3.4 Rule 3: attach the weather series

The weather series carries one value per logger stamp, assigned by nearest hour when the series was retrieved. Within each 30-minute bin the script keeps the value carried by the :01/:31 stamps, the cadence of most loggers, for which a :31 stamp rounds to the following hour (the few loggers on a :00/:30 cadence carry the even hour at :30, pandas' round-half-to-even, and their rows are used only in a bin without a :01/:31 stamp, which happens once, before any analysed night):

Hide

```
wx <- wx_raw %>% mutate(datetime = read_stamps(datetime), time_bin = round_date(datetime, "30
minutes"),
                        pref = as.integer(minute(datetime) %% 30 != 1)) %>% # :01/:31 rows first
arrange(region, time_bin, pref) %>% distinct(region, time_bin, .keep_all = TRUE) %>%
transmute(region, time_bin, wind_kmh = wind, cloud_pct = cloud)
```

3.5 Rules 4 to 6: which nights, which loggers

Rule	Definition	Constant	Origin
calm & clear night	ECMWF 10 m wind < 10 km h ⁻¹ AND total cloud < 25 % in at least 75 % of the 24 night bins	WIND_MAX_KMH, CLOUD_MAX_PCT, MIN_FRAC_STEPS	study-specific thresholds on the ECMWF analysis; the calm/clear requirement itself follows Oke (1982) and Stewart & Oke (2012)
complete weather series	all 24 weather bins of the night present (partial first/last nights are not classified)		study-specific
network completeness	at least 20 loggers of the district reporting on the night (a median of a handful of loggers is not a district reference)	MIN_SENSORS_PER_NIGHT	study-specific
sensor-night completeness	a sensor-night is kept only if at least 18 of its 24 bins are present	MIN_STEPS_PER_SENSOR_NIGHT	study-specific

Every rule is a named constant in `helpers.R`, so the threshold sensitivity of Section 4.2 can vary them. Running the whole step:

Hide

```
b <- load_binned(); hh <- b$hh; wx <- b$wx
inv <- night_inventory(hh, wx) # one row per district x diurnal date
sn <- sensor_nights(hh, inv) # one row per logger x selected night
inv %>% count(region, decision) %>% pivot_wider(names_from = region, values_from = n, values_fill = 0)
%>% tab()
```

decision	Ewa	Honolulu
not calm and clear	38	49
partial night (deployment/retrieval)	1	1
selected	4	7
fewer than 20 sensors reporting	0	1

Hide

```
inv %>% filter(selected) %>% select(region, night_date, n_sensors, wind_night, cloud_night,
frac_calm_clear) %>%
  mutate(across(where(is.numeric), ~ round(.x, 2))) %>% tab(caption = "The eleven analysed nights")
```

The eleven analysed nights

region	night_date	n_sensors	wind_night	cloud_night	frac_calm_clear
Ewa	2024-11-25	38	6.12	11.54	0.75
Ewa	2024-12-01	38	1.89	16.29	0.75
Ewa	2024-12-21	38	2.69	9.79	0.88
Ewa	2024-12-25	38	2.61	15.42	0.75
Honolulu	2024-11-30	35	6.92	14.75	0.75
Honolulu	2024-12-01	35	1.71	5.21	0.92
Honolulu	2024-12-07	35	5.99	20.25	0.79
Honolulu	2024-12-16	36	3.20	6.96	0.92
Honolulu	2024-12-21	36	3.38	5.29	1.00
Honolulu	2024-12-26	36	1.61	8.00	0.92
Honolulu	2025-01-03	36	3.28	7.46	0.96

Hide

```
sn %>% group_by(region) %>% summarise(sites = n_distinct(sensor_id), nights = n_distinct(night_date),
sensor_nights = n()) %>% tab()
```

region	sites	nights	sensor_nights
Ewa	38	4	152
Honolulu	36	7	248

The 400 sensor-nights (36 Honolulu sites × 7 nights minus four incomplete sensor-nights; 38 'Ewa sites × 4 nights) are the unit of analysis. Their night-mean `dT_night` is what the models explain. The step writes `data/processed/night_inventory.csv`, `sensor_nights.csv` and `halfhourly_calm_clear.csv` (the full diurnal cycles that contain an analysed night, used by the figures and the shared-night comparison).

4 Step 2: describing each site (01_site_predictors.R)

Eight descriptors are computed for a circle of radius 50, 100 and 200 m around each logger, all from public data and all in the projection of the C-CAP tiles (NAD83(PA11) / UTM 4N, EPSG:6634, metres):

Descriptor	Source	How	Package
impervious, tree-canopy and water fractions (%)	NOAA C-CAP 2021, 1 m masks	share of the 1 m pixels whose centre lies inside the circle that carry the class value	terra: <code>vrt()</code> builds one virtual mosaic per mask from the tiles; <code>extract(touches = FALSE)</code> returns the pixels inside the polygon
building footprint fraction (%)	FEMA / ORNL USA Structures	area of footprints clipped to the circle ÷ circle area	sf: <code>st_intersects()</code> to find candidates, <code>st_intersection()</code> to clip, <code>st_area()</code>

Descriptor	Source	How	Package
mean building height (m)	FEMA (height attribute; 5 m where the inventory records none, clipped to 0.5–60 m)	footprint-area-weighted mean of the clipped footprints	sf + base
canyon aspect ratio H/W	FEMA	mean height ÷ mean building spacing, $\text{spacing} = \sqrt{(\text{circle area} / n)}$ – $\sqrt{(\text{mean footprint})}$, $W \geq 6$ m, $H/W \leq 5$	base
sky view factor (point)	FEMA footprints	36 rays from the sensor at 1.5 m out to 200 m; along each ray the highest elevation angle β of a footprint; $SVF = \text{mean } \cos^2 \beta$ (Johnson & Watson 1984)	sf: <code>st_linestring()</code> , <code>st_intersection()</code> , <code>st_distance()</code>
distance to coast (km)	GSHHG full-resolution shoreline	distance from the sensor to the boundary of the island polygon (Pearl Harbor counts as coast)	sf: <code>st_union()</code> , <code>st_boundary()</code> , <code>st_distance()</code>

The functions are short enough to read in full. The raster fraction:

Hide

```
raster_fraction <- function(r, geom, values) {
  v <- terra::extract(r, vect(geom), touches = FALSE)[[2]]      # every 1 m pixel whose centre is
  inside the circle
  n <- length(v)                                                  # no-data (0) pixels are read as NA and
  count towards n
  list(frac = setNames(vapply(values, function(k) 100 * sum(v == k, na.rm = TRUE) / n, numeric(1)),
    values), n = n)
}
```

The sky view factor, which is the one genuinely three-dimensional descriptor:

Hide

```
sky_view_factor <- function(pt, bldg, z0 = 1.5, R = 200, n_az = 36) {
  cand <- bldg[st_intersects(st_buffer(pt, R), bldg)[[1]], ]      # footprints within
  the horizon
  xy <- st_coordinates(pt); total <- 0
  for (k in seq_len(n_az) - 1) {
    th <- 2 * pi * k / n_az
    ray <- st_sfc(st_linestring(rbind(xy, xy + R * c(sin(th), cos(th)))), crs = st_crs(pt))
    hit <- cand[st_intersects(ray, cand)[[1]], ]; beta <- 0
    for (j in seq_len(nrow(hit))) {
      seg <- st_intersection(ray, st_geometry(hit)[j])            # where the ray enters
      the footprint
      d <- max(1, as.numeric(st_distance(pt, seg)))                # horizontal distance
      to that wall
      beta <- max(beta, atan(max(hit$height_m[j] - z0, 0) / d))    # elevation angle of
      the wall top
    }
    total <- total + cos(beta)^2
  }
  total / n_az
}
```

The full computation takes about seven minutes (the building layer holds every footprint on O‘ahu), so this document reads the result rather than recomputing it, and demonstrates the machinery on one site:

Hide

```

source(file.path(root, "analysis", "01_site_predictors.R"), encoding = "UTF-8")      # function
definitions only
sites <- read_csv(file.path(PROC, "sensor_nights.csv"), show_col_types = FALSE) %>% distinct(sensor_id,
.keep_all = TRUE) # coordinates come with the stored table
pts <- st_transform(st_as_sf(sites, coords = c("longitude", "latitude"), crs = 4326, remove = FALSE),
CRS_M)
p1 <- pts[pts$sensor_id == "HNL50", ] # the compact high-rise
site of Waikiki
circle <- st_buffer(st_geometry(p1), 100, nQuadSegs = 16)
bldg <- buildings_near(st_geometry(p1), 300) # footprints within 300 m,
heights filled as in the script
imp <- raster_fraction(ccap_mask("impervious"), circle, LANDCOVER$impervious$value)
bm <- building_metrics(circle, bldg)
svf <- sky_view_factor(st_geometry(p1), bldg)
cat(sprintf("HNL50 at 100 m: impervious %.1f %% of %d pixels; footprint %.1f %%; mean height %.1f m;
H/W %.2f; SVF at the sensor %.2f\n",
imp$frac[["1"]], imp$n, bm$bldg_frac, bm$zH, bm$hw_ratio, svf))

```

```

#> HNL50 at 100 m: impervious 60.4 % of 31378 pixels; footprint 19.3 %; mean height 6.8 m; H/W 0.39;
SVF at the sensor 0.07

```

Hide

```

pred <- read_csv(file.path(DER, "site_predictors_multiscale.csv"), show_col_types = FALSE)
pred %>% filter(sensor_id == "HNL50") %>% select(radius_m, imperv, tree, bldg, height, aspect,
svf_point, coast_km) %>%
mutate(across(-radius_m, ~ round(.x, 2))) %>% tab(caption = "The same site in the stored table (all
three radii)")

```

The same site in the stored table (all three radii)

radius_m	imperv	tree	bldg	height	aspect	svf_point	coast_km
50	59.50	0.68	20.31	6.05	0.39	0.07	1.7
100	60.35	12.20	19.32	6.79	0.39	0.07	1.7
200	76.17	11.46	25.54	7.83	0.47	0.07	1.7

HNL50 is also one of the five recorded positions that fall inside a building footprint, which is why the ray casting returns a sky view factor near zero for it; Section 4.2 recomputes those five values with the position moved outside the footprint and shows that nothing depends on them.

Hide

```

pred %>% filter(radius_m == 100) %>% group_by(region) %>%
summarise(across(all_of(PRED), ~ round(mean(.x), 2))) %>% tab(caption = "District means of the eight
descriptors at 100 m")

```

District means of the eight descriptors at 100 m

region	imperv	tree	bldg	water	height	svf_point	aspect	coast_km
Ewa	52.98	9.89	12.27	0.45	4.69	0.96	0.18	2.57
Honolulu	62.56	11.86	20.37	3.99	8.93	0.84	0.45	1.33

The step writes `data/descriptors/site_predictors_multiscale.csv`: one row per site and radius.

5 Step 3: the mixed-effects models (02_models.R)

5.1 Why a mixed model, and how it is written in lme4

A site contributes several nights, so its sensor-nights are not independent: whatever makes a site warm on one calm night tends to make it warm on the next. A linear mixed model with a random intercept per site handles this directly. In `lme4` syntax the full pooled model is

```
dT_night ~ region + imperv_z + tree_z + bldg_z + water_z + height_z + svf_point_z + aspect_z +
coast_km_z + (1 | sensor_id)
```

where `(1 | sensor_id)` is the site random intercept, `region` is the district ('Ewa as reference level) and the `_z` predictors are the descriptors standardised with the *pooled* mean and SD, so that one coefficient means “°C per one standard deviation” in every model and is comparable between districts. The wrapper used throughout:

Hide

```
fit_lmm <- function(df, rhs, reml = TRUE) {
  f <- as.formula(paste0("dT_night ~ ", rhs, " + (1 | sensor_id)"))
  lmerTest::lmer(f, data = df, REML = reml,
    control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun = 1e5),
    check.conv.singular = "ignore"))
}
```

Two conventions run through every step, following Zuur et al. (2009) and Bates et al. (2015):

- **REML for estimates, ML for tests.** Coefficients, standard errors and variance components are reported from REML fits (unbiased variance components). Models that differ in their fixed effects are compared with likelihood-ratio tests and AIC on ML fits, because REML likelihoods of models with different fixed effects are not comparable.
- **Satterthwaite p-values for individual coefficients** (`lmerTest`), which account for the fact that a site-level predictor is effectively tested on 74 sites, not 400 sensor-nights.

5.2 The building blocks, run live

Hide

```
source(file.path(root, "analysis", "02_models.R"), encoding = "UTF-8") # helper functions; main()
is not run
d <- load_radius(100)$d
z <- zscore_frame(d, PRED); d <- z$df; sds <- z$sds
```

Null model and intraclass correlation. How much of the variance of `dT_night` is between sites rather than between nights within a site?

Hide

```
m0 <- fit_lmm(d, "1", reml = TRUE)
VarCorr(m0)
```

```
#> Groups      Name      Std.Dev.
#> sensor_id (Intercept) 0.63500
#> Residual              0.49059
```

Hide

```
vc <- var_comp(m0); cat(sprintf("ICC = %.3f / (%.3f + %.3f) = %.2f\n", vc$var_site, vc$var_site,
vc$var_resid, vc$var_site / (vc$var_site + vc$var_resid)))
```

```
#> ICC = 0.403 / (0.403 + 0.241) = 0.63
```

Hide

```
performance::icc(m0)
```

ICC_adjusted <dbl>	ICC_unadjusted <dbl>	optional <lgl>
0.6262238	0.6262238	FALSE

1 row

The full pooled model, as `summary()` prints it:

Hide

```
m_full <- fit_lmm(d, rhs_full("region"), reml = TRUE)
summary(m_full)
```

```
#> Linear mixed model fit by REML. t-tests use Satterthwaite's method [
#> lmerModLmerTest]
#> Formula: f
#> Data: df
#> Control: ctrl
#>
#> REML criterion at convergence: 728.8
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.5291 -0.3919 -0.0506  0.3564  6.9153
#>
#> Random effects:
#> Groups      Name      Variance Std.Dev.
#> sensor_id (Intercept) 0.2753   0.5247
#> Residual              0.2402   0.4901
#> Number of obs: 400, groups:  sensor_id, 74
#>
#> Fixed effects:
#>              Estimate Std. Error      df t value Pr(>|t|)
#> (Intercept)    0.23074    0.11951  67.90811    1.931 0.057694 .
#> regionHonolulu -0.33825    0.17731  64.87463   -1.908 0.060864 .
#> imperv_z        0.40336    0.11705  65.74712    3.446 0.000996 ***
#> tree_z          -0.14300    0.07502  65.18761   -1.906 0.061040 .
#> bldg_z          -0.25449    0.13976  65.19763   -1.821 0.073214 .
#> water_z         0.11097    0.08609  62.18391    1.289 0.202164
#> height_z        0.27688    0.10669  63.41885    2.595 0.011731 *
#> svf_point_z     -0.03097    0.07555  62.10405   -0.410 0.683234
#> aspect_z        -0.08128    0.14469  63.19262   -0.562 0.576280
#> coast_km_z      -0.19639    0.07963  66.75451   -2.466 0.016227 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>              (Intr)  rgnHnl  imprv_  tree_z  bldg_z  watr_z  hght_z  svf_p_  aspc_
#> regionHonl1 -0.829
#> imperv_z    -0.009  0.015
#> tree_z      0.070 -0.078  0.248
#> bldg_z      0.030 -0.037 -0.608 -0.084
#> water_z     0.197 -0.210  0.347  0.218 -0.026
#> height_z    0.166 -0.181 -0.130 -0.196  0.223  0.016
#> svf_point_z -0.193  0.207  0.121 -0.017  0.031 -0.011 -0.009
#> aspect_z    0.084 -0.090  0.119  0.220 -0.570  0.024 -0.606  0.012
#> coast_km_z  -0.343  0.369  0.015  0.205  0.151  0.203  0.017  0.112  0.010
```

Explained variance (Nakagawa & Schielzeth 2013). R^2_m is the share of the total variance explained by the fixed effects; R^2_c adds the site random intercepts:

Hide

```
r2 <- r2_nakagawa(m_full); unlist(r2[c("r2m", "r2c", "icc_cond")])
```

```
#>      r2m      r2c  icc_cond
#> 0.2541226 0.6524692 0.5340645
```

Hide

```
performance::r2_nakagawa(m_full)
```

```
#> # R2 for Mixed Models
#>
#> Conditional R2: 0.652
#> Marginal R2: 0.254
```

Likelihood-ratio test of the eight predictors given district, on ML fits (this is what `anova()` does for two nested `lmer` fits):

Hide

```
m_full_ml <- fit_lmm(d, rhs_full("region"), reml = FALSE)
m_reg_ml <- fit_lmm(d, "region", reml = FALSE)
anova(m_reg_ml, m_full_ml, refit = FALSE)
```

	npar <dbl>	AIC <dbl>	BIC <dbl>	logLik <dbl>	deviance <dbl>	Chisq <dbl>	Df <dbl>	Pr(>Chisq) <dbl>
m_reg_ml	4	740.7611	756.727	-366.3806	732.7611	NA	NA	NA
m_full_ml	12	721.8434	769.741	-348.9217	697.8434	34.91771	8	2.767722e-05

2 rows

Natural units. A slope per standard deviation is converted back with the pooled SD, e.g. the impervious effect per +10 percentage points:

Hide

```
ct <- coef_table(m_full, sds)
ct %>% filter(term %in% c("imperv_z", "height_z", "coast_km_z")) %>% select(term, estimate, se, df, p,
estimate_natural, se_natural, natural_unit) %>%
mutate(across(where(is.numeric), ~ signif(.x, 3))) %>% tab()
```

term	estimate	se	df	p	estimate_natural	se_natural	natural_unit
imperv_z	0.403	0.1170	65.7	0.000996	0.1610	0.0466	per +10 pp
height_z	0.277	0.1070	63.4	0.011700	0.0561	0.0216	per +1 m
coast_km_z	-0.196	0.0796	66.8	0.016200	-0.1770	0.0718	per +1 km

5.3 The sequence of analyses in 02_models.R

The script runs thirteen steps; each writes a CSV table to `results/tables/` and a block of `results/thesis_results.json`. What each one does, and the thesis table, figure and section it feeds:

Step	What	Method	Thesis
1	descriptives of the predictors by district and radius; district differences	means, SDs; Mann-Whitney (<code>wilcox.test()</code>)	Table 4, §4.1
2	scale selection	full model at 50/100/200 m, pooled and per district, ML AIC; the radius that is best or within 2 AIC in all three (minimax Δ AIC) is adopted; a forward-stepwise OLS on site means (<code>lm()</code> + <code>AIC()</code>) as a secondary check	Table 6, Fig. 13, §4.3

Step	What	Method	Thesis
3	collinearity	site-level Pearson correlations; VIF from <code>car::vif()</code>	Table 7, Fig. 14, §4.4
4	null models	intercept-only LMMs, REML; ICC	Table 5, §4.2
5	single-predictor models	one predictor at a time, district-adjusted and per district; LRT vs null (ML), R^2_m	Table 8, Fig. 15, §4.5
6	full pooled model and parsimonious model	REML estimates, Satterthwaite p, LRTs; predicted district gap at each district's mean morphology	Table 9, Fig. 16, §4.6
7	district models and best subsets	full 8-predictor model per district; all 93 subsets of ≤ 3 predictors ranked by ML AIC; Akaike weights and predictor importance	Tables 10–12, §4.7
8	district $\times$ predictor interactions	each interaction added singly to the full model, LRT; joint test of all eight	Table 13, Fig. 17, §4.7
9	night effect and night-to-night stability	night as fixed effect, LRT; correlation of site ΔT between nights	Fig. 12, §4.1–4.2
10	threshold sensitivity	night selection re-run from the record under 24 rules; models refitted; complement (windy/cloudy) nights	Table 15, Fig. 18, §4.8
11	model comparison	ML AIC of OLS, null, district, parsimonious, full and interaction models	Table 14
12	site effects, leave-one-site-out CV, calm-core refit	<code>raneef(condVar = TRUE)</code> ; each site predicted from a model fitted without it (<code>predict(re.form = NA)</code>); refit on nights with wind $< 5 \text{ km h}^{-1}$	Table 17, Figs 20 & 21, §4.8–4.9
13	shared nights	absolute night-mean medians and percentile bands of both networks on 1 and 21 December	Table 18, Fig. 22, §4.10

Three of these deserve a closer look because they are where a reader is most likely to ask “how exactly?”

5.3.1 Best subsets and Akaike weights (step 7)

Every combination of at most three of the eight predictors (93 models including the null) is fitted by ML within each district and ranked by AIC. The Akaike weight of a model is its relative likelihood, $\exp(-\Delta\text{AIC}/2)$ normalised over the set; the *importance* of a predictor is the sum of the weights of the models that contain it. This is exactly what `MuMIn::dredge()` would produce; it is written out here so that nothing is hidden:

Hide

```
cands <- c(list(character(0)), unlist(lapply(1:3, function(k) combn(PRED, k, simplify = FALSE)),
recursive = FALSE))
rr <- bind_rows(lapply(cands, function(cc) {
  rhs <- if (length(cc)) paste(paste0(cc, "_z"), collapse = " + ") else "1"
  mm <- fit_lmm(g, rhs, reml = FALSE)
  tibble(predictors = if (length(cc)) paste(cc, collapse = " + ") else "(null)", k = length(cc), aic =
AIC(mm))
})) %>% mutate(delta_aic = aic - min(aic), weight = exp(-0.5 * delta_aic) / sum(exp(-0.5 * delta_aic)))
%>% arrange(aic)
```

Hide

```
res <- fromJSON(file.path(OUT, "thesis_results.json"), simplifyVector = TRUE)
bind_rows(lapply(REGIONS, function(r) as_tibble(res$regional[[r]]$best_subset$top5) %>% mutate(region =
r) %>% head(3))) %>%
  select(region, predictors, k, delta_aic, weight, r2m) %>% mutate(across(where(is.numeric), ~
round(., 2))) %>%
  tab(caption = "The three best subsets per district")
```

The three best subsets per district

region	predictors	k	delta_aic	weight	r2m
Honolulu	imperv + coast_km	2	0.00	0.18	0.36
Honolulu	imperv + bldg + coast_km	3	0.09	0.17	0.38
Honolulu	imperv + aspect + coast_km	3	1.61	0.08	0.36
Ewa	imperv + height + aspect	3	0.00	0.66	0.31
Ewa	imperv + tree + height	3	3.38	0.12	0.29
Ewa	imperv + bldg + height	3	5.22	0.05	0.28

5.3.2 Threshold sensitivity (step 10)

Because the night-selection thresholds are study-specific choices, the whole selection is re-run from the binned record under a grid of rules (wind 8–20 km h⁻¹, cloud 15–50 %, bin share 60–100 %, and no weather filter at all), and the parsimonious and full pooled models are refitted each time. The only thing that changes between runs is the arguments of `night_inventory()` :

Hide

```
inv <- night_inventory(hh, wx, wind_max = w, cloud_max = cc, min_frac = f)
sn <- sensor_nights(hh, inv)
```

Hide

```
as_tibble(res$threshold_sensitivity) %>% filter(min_frac == MIN_FRAC_STEPS & cloud_max == CLOUD_MAX_PCT
| wind_max %in% c(999, -1)) %>%
  select(wind_max, cloud_max, nights_honolulu, nights_ewa, sensor_nights, beta_imperv_z, beta_height_z,
r2m_full, corr_with_baseline) %>%
  mutate(across(where(is.numeric), ~ round(.x, 2))) %>% tab(caption = "Wind threshold varied at cloud <
25 %; 999 = no weather filter, -1 = windy/cloudy nights only")
```

Wind threshold varied at cloud < 25 %; 999 = no weather filter, -1 = windy/cloudy nights only

wind_max	cloud_max	nights_honolulu	nights_ewa	sensor_nights	beta_imperv_z	beta_height_z	r2m_full	corr_with_baseline
8	25	5	4	331	0.24	0.20	0.28	0.99
10	25	7	4	400	0.22	0.19	0.25	1.00
12	25	7	7	514	0.21	0.19	0.24	0.99
15	25	9	11	737	0.20	0.18	0.22	0.98
20	25	11	14	923	0.18	0.17	0.21	0.97
999	999	42	42	3087	0.14	0.13	0.13	0.96
-1	-1	35	38	2687	0.13	0.12	0.12	0.95

5.3.3 Leave-one-site-out cross-validation (step 12)

Each of the 74 sites is held out in turn, the model is refitted on the other sites, and the held-out site's mean ΔT is predicted from the fixed effects alone (`predict(m, newdata, re.form = NA)`), i.e. from its morphology only. This is the honest measure of how well the descriptors would predict a site that was never measured.

Hide

```
as_tibble(res$loso_cv) %>% select(model, rmse, mae, r, r2_cv) %>% mutate(across(where(is.numeric), ~
round(.x, 2))) %>% tab()
```

model	rmse	mae	r	r2_cv
full	0.63	0.49	0.39	0.09
parsimonious	0.63	0.49	0.36	0.11
region only	0.68	0.53	-0.47	-0.05
Honolulu full	0.64	0.53	0.54	0.22
Honolulu best subset (imperv + coast_km)	0.59	0.47	0.58	0.33
Ewa full	0.47	0.39	0.65	0.41
Ewa best subset (imperv + height + aspect)	0.44	0.36	0.71	0.49

5.3.4 Absolute comparison on the shared nights (step 13)

Because ΔT is measured from each district's own median, the model terms say nothing about which district is warmer. Step 13 answers that on the two calm, clear nights the deployments share (1 and 21 December 2024) from the half-hourly record itself: the network median and the 10th–90th percentile band of each district through the diurnal cycle (thesis Table 18 and Figure 22).

Hide

```
read_csv(file.path(TAB, "shared_nights.csv"), show_col_types = FALSE) %>%
  select(night_date, region, n_sensors, T_median_night, T_p10, T_p90, wind, cloud) %>%
  mutate(across(where(is.numeric), ~ round(.x, 2))) %>% tab(caption = "Night-mean (18:00–06:00)
statistics of the two networks on the shared nights")
```

Night-mean (18:00–06:00) statistics of the two networks on the shared nights

night_date	region	n_sensors	T_median_night	T_p10	T_p90	wind	cloud
2024-12-01	Honolulu	35	21.70	21.01	22.71	1.71	5.21
2024-12-01	Ewa	38	20.05	18.88	21.08	1.89	16.29
2024-12-21	Honolulu	36	22.01	21.15	23.22	3.38	5.29
2024-12-21	Ewa	38	20.87	19.83	21.87	2.69	9.79

6 Step 4: the checks (06_elevation_check.R, 07_sensitivity_checks.R)

6.1 Terrain (06_elevation_check.R)

In Honolulu the strongest single descriptor is distance to the coast, and the shore-to-valley gradient it describes also runs up-slope, so a reader may ask whether the “coast” effect is really an elevation effect.

06_elevation_check.R answers that with the same machinery: the bare-earth elevation of each logger position (USGS 3DEP 1 m lidar DEM, read through the USGS Elevation Point Query Service and stored with its source raster id in data/descriptors/site_elevation.csv) is z-scored like the other descriptors and added to the models. The decisive comparison, run live, is coast alone against coast plus elevation in Honolulu, with `anova()` giving the likelihood-ratio test on ML fits and `summary()` the Satterthwaite *t* tests of the REML fit:

Hide

```
el <- read_csv(file.path(DER, "site_elevation.csv"), show_col_types = FALSE) %>% select(sensor_id,
elev_m)
dh <- d %>% inner_join(el, by = "sensor_id") %>% mutate(elev_m_z = (elev_m - mean(elev_m)) /
sd(elev_m)) %>% # pooled z-score, as for the other descriptors
  filter(region == "Honolulu")
m_coast <- fit_lmm(dh, "coast_km_z", reml = FALSE)
m_both <- fit_lmm(dh, "coast_km_z + elev_m_z", reml = FALSE)
m_elev <- fit_lmm(dh, "elev_m_z", reml = FALSE)
anova(m_coast, m_both, refit = FALSE) # does elevation add anything given the coast?
```

	npar <dbl>	AIC <dbl>	BIC <dbl>	logLik <dbl>	deviance <dbl>	Chisq <dbl>	Df <dbl>	Pr(>Chisq) <dbl>
m_coast	4	294.4125	308.4662	-143.2063	286.4125	NA	NA	NA
m_both	5	296.3581	313.9253	-143.1791	286.3581	0.05437586	1	0.8156167

2 rows

Hide

```
anova(m_elev, m_both, refit = FALSE) # does the coast add anything given elevation?
```

	npar <dbl>	AIC <dbl>	BIC <dbl>	logLik <dbl>	deviance <dbl>	Chisq <dbl>	Df <dbl>	Pr(>Chisq) <dbl>
m_elev	4	303.1021	317.1558	-147.5510	295.1021	NA	NA	NA
m_both	5	296.3581	313.9253	-143.1791	286.3581	8.743905	1	0.003106386

2 rows

Hide

```
summary(fit_lmm(dh, "coast_km_z + elev_m_z", reml = TRUE))$coefficients
```

```
#>           Estimate Std. Error      df    t value    Pr(>|t|)
#> (Intercept) -0.23013646  0.1230278 32.89192 -1.8706050 0.070320075
#> coast_km_z  -0.60883411  0.2023147 32.88911 -3.0093420 0.004995146
#> elev_m_z    -0.03283542  0.1475149 32.75595 -0.2225906 0.825236330
```

Given distance to the coast, elevation adds nothing; given elevation, the coast remains. The script repeats this for the 'Ewa and pooled models, for the best-subset and full models (thesis Table 16), re-runs the exhaustive best-subset search with elevation among the nine candidates, and re-runs the leave-one-site-out cross-validation with and without it:

Hide

```
em <- read_csv(file.path(TAB, "elevation_models.csv"), show_col_types = FALSE)
em %>% filter(scope != "pooled", grepl("elevation|coast only|best subset", model)) %>%
  select(scope, model, r2m, coast_per_km, coast_p, elev_per_10m, elev_p, lrt_elev_given_model_p,
delta_aic_vs_without) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>% tab(caption = "District models with and without
site elevation (natural units: °C per km, °C per 10 m)")
```

District models with and without site elevation (natural units: °C per km, °C per 10 m)

scope	model	r2m	coast_per_km	coast_p	elev_per_10m	elev_p	lrt_elev_given_model_p	delta_aic_vs_without
Honolulu	elevation only	0.112	NA	NA	-0.315	0.026	NA	NA
Honolulu	coastonly	0.258	-0.574	0.000	NA	NA	NA	NA

scope	model	r2m	coast_per_km	coast_p	elev_per_10m	elev_p	lrt_elev_given_model_p	delta_aic_vs_without
Honolulu	coast + elevation	0.255	-0.549	0.005	-0.034	0.825	NA	NA
Honolulu	bestsubset (imperv + coast)	0.342	-0.615	0.000	NA	NA	NA	NA
Honolulu	bestsubset (imperv + coast) + elevation	0.339	-0.575	0.002	-0.057	0.692	0.671	1.820
Honolulu	full (eight descriptors) + elevation	0.388	-0.492	0.011	-0.061	0.683	0.625	1.761
Ewa	elevation only	0.005	NA	NA	-0.076	0.556	NA	NA
Ewa	coastonly	0.001	-0.017	0.843	NA	NA	NA	NA
Ewa	coast + elevation	0.005	0.022	0.841	-0.096	0.560	NA	NA
Ewa	bestsubset (imperv + height + aspect)	0.305	NA	NA	NA	NA	NA	NA
Ewa	bestsubset (imperv + height + aspect) + elevation	0.307	NA	NA	-0.070	0.442	0.406	1.309
Ewa	full (eight descriptors) + elevation	0.330	-0.013	0.867	-0.093	0.394	0.316	0.996

[Hide](#)

```
ec <- fromJSON(file.path(OUT, "elevation_check.json"), simplifyVector = FALSE)
cat(sprintf("Best subset with elevation in the pool. Honolulu: %s (elevation importance %.2f); 'Ewa: %s (%.2f)\n",
            ec$subsets$Honolulu$best, ec$subsets$Honolulu$elevation_importance, ec$subsets$Ewa$best,
            ec$subsets$Ewa$elevation_importance))
```

```
#> Best subset with elevation in the pool. Honolulu: imperv + coast_km (elevation importance 0.11);
'Ewa: imperv + height + aspect (0.02)
```

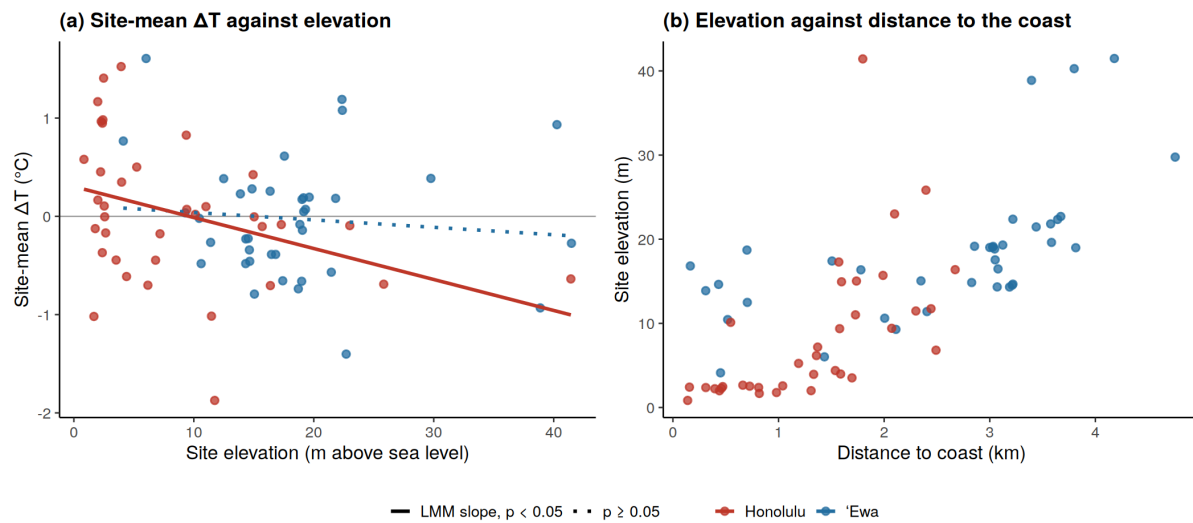
[Hide](#)

```
as_tibble(bind_rows(lapply(ec$loso, as_tibble))) %>% filter(grepl("best subset", model)) %>%
mutate(across(where(is.numeric), ~ round(.x, 3))) %>% tab(caption = "Leave-one-site-out skill of the
best-subset models with and without elevation")
```

Leave-one-site-out skill of the best-subset models with and without elevation

scope	model	rmse	mae	r	r2_cv
Honolulu	bestsubset (imperv + coast)	0.592	0.474	0.578	0.326
Honolulu	bestsubset (imperv + coast) + elevation	0.599	0.481	0.566	0.309

scope	model	rmse	mae	r	r2_cv
Ewa	best subset (imperv + height + aspect)	0.435	0.364	0.705	0.491
Ewa	best subset (imperv + height + aspect) + elevation	0.451	0.372	0.682	0.454



6.2 Three site-level checks (07_sensitivity_checks.R)

The second check script asks whether any conclusion rests on a single site or on a data-quality detail. It refits the models of 02_models.R (same z-scores, same random-intercept structure, likelihood-ratio tests on ML fits, REML estimates) under three changes and writes results/sensitivity_checks.json and results/tables/sensitivity_*.csv:

- **A. HNL08.** The warmest Honolulu site is also among the most variable and is not explained by its descriptors; a generator next to the station was likely the source of its extra warmth (thesis Section 5.4). The models are refitted without its seven sensor-nights, and the check reports how far the Honolulu network median moves when the logger is left out of it.
- **B. Buildings without a recorded height.** About a quarter of the O'ahu footprints have no height in the USA Structures inventory and carry a 5 m fill value in the footprint file. For each site the share of building area within 100 m that carries the fill value is computed (data/descriptors/site_default_height_share.csv); the models are refitted with 3 m instead of 5 m for those buildings (height and canyon aspect ratio recomputed at 100 m) and without the six Ewa sites where such buildings make up more than half of the building area.
- **C. Sensor positions inside footprints.** Five recorded positions fall inside a footprint polygon (a post under an eave or a covered walkway, or a position error of a few metres), so the ray casting sees a wall 1 m away in every direction and returns a sky view factor near zero. Their sky view factor is recomputed with the position moved to 1 m outside the nearest footprint edge (data/descriptors/site_svf_footprint_check.csv), and the models are refitted.

Hide

```

sc <- fromJSON(file.path(OUT, "sensitivity_checks.json"), simplifyVector = FALSE)
pick <- function(x, what) { cf <- x$pooled_full$coefs; tibble(variant = what, n_sites = x$n_sites, r2m
= x$pooled_full$r2m,
  imperv_per_10pp = cf$imperv$est, imperv_p = cf$imperv$p, height_per_m = cf$height$est, height_p =
cf$height$p,
  coast_per_km = cf$coast_km$est, coast_p = cf$coast_km$p) }
bind_rows(pick(sc$hnl08$with, "all sites (thesis)"), pick(sc$hnl08$without, "A: without HNL08"),
  pick(sc$default_heights$fill_3m, "B: 3 m for buildings without a recorded height"),
  pick(sc$default_heights$without, "B: without the six sites dominated by them"),
  pick(sc$svf_footprints$corrected, "C: sky view factor recomputed for the five positions"))
%>%
  mutate(across(where(is.numeric), ~ signif(.x, 3))) %>% tab(caption = "The full pooled model under
each check (natural units)")

```

The full pooled model under each check (natural units)

variant	n_sites	r2m	imperv_per_10pp	imperv_p	height_per_m	height_p	coast_per_km	coast_p
all sites (thesis)	74	0.254	0.161	0.000996	0.0561	0.0117	-0.177	0.01620
A: without HNL08	73	0.290	0.159	0.000500	0.0512	0.0133	-0.176	0.01020
B: 3 m for buildings without a recorded height	74	0.253	0.165	0.000756	0.0556	0.0133	-0.176	0.01730
B: without the six sites dominated by them	68	0.254	0.146	0.005400	0.0556	0.0144	-0.217	0.00635
C: sky view factor recomputed for the five positions	74	0.258	0.158	0.001160	0.0550	0.0130	-0.181	0.01380

Hide

```

as_tibble(bind_rows(lapply(sc$svf_footprints$sites, as_tibble))) %>% mutate(across(where(is.numeric), ~
round(.x, 2))) %>%
  tab(caption = "The five positions inside footprints: sky view factor as recorded and after moving the
position 1 m outside the footprint")

```

The five positions inside footprints: sky view factor as recorded and after moving the position 1 m outside the footprint

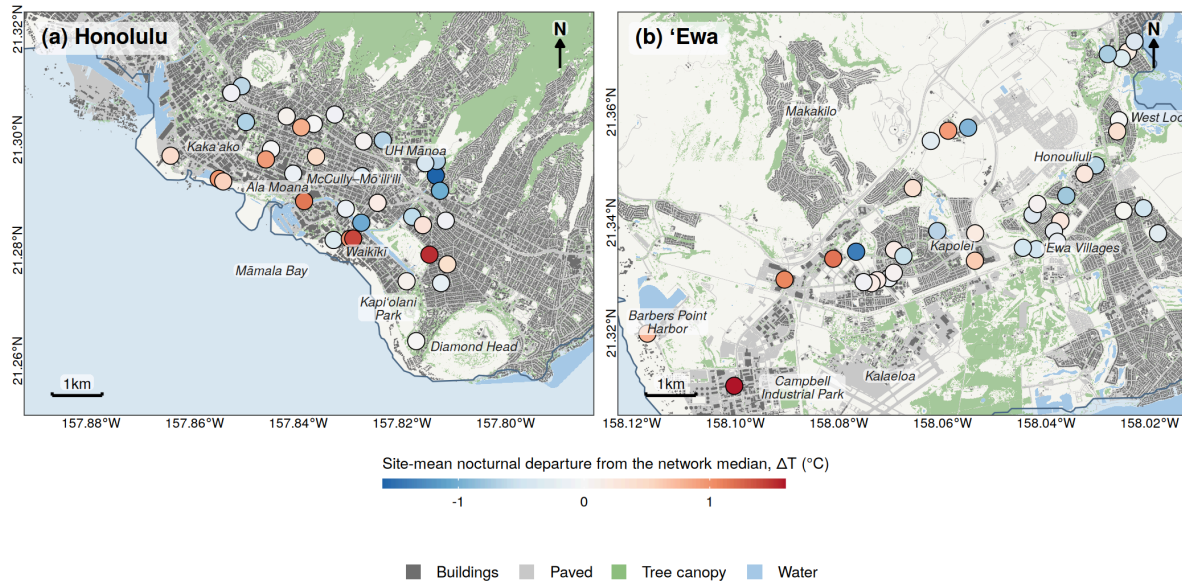
sensor_id	region	footprint_height_m	distance_to_edge_m	moved_inside	svf_moved	svf_original
EWA14	Ewa	6.21	13.19	FALSE	0.61	0.04
HNL04	Honolulu	4.90	1.08	FALSE	0.65	0.08
HNL21	Honolulu	9.94	5.02	FALSE	0.52	0.01
HNL43	Honolulu	6.74	0.69	FALSE	0.54	0.04
HNL50	Honolulu	5.02	0.44	FALSE	0.61	0.07

None of the three changes alters a conclusion: the impervious and building-height effects keep their signs and significance, the district difference in the coastal effect remains, and the 'Ewa height effect is 0.135 rather than 0.131 °C per metre with 3 m fill heights.

7 Step 5: figures and tables (03_figures.R, 05_thesis_tables.R,

08_appendix_figures.R

Every figure is a ggplot2 object saved as PNG (300 dpi) and PDF in `results/figures/` (the repository keeps the PDF versions of the statistical figures only; the vector maps run to 10 MB each and are regenerated by the script). The maps are built from the same inputs as the descriptors: `terra` crops each C-CAP mask to the district box and aggregates it to 4 m (`aggregate(fact = 4, fun = "mean")`), cached as GeoTIFF), `sf` supplies the island polygon, the building footprints and the site points, and `ggplot2::coord_sf()` draws the graticule. The observed pattern, as an example (thesis Figure 11):



`05_thesis_tables.R` reads `thesis_results.json` and the CSV tables and writes 21 of the thesis's 22 tables (all but the descriptor definitions) as Markdown files in `results/thesis_tables/`, with the captions and the rounding used in the document, so that every printed number can be traced to a line of code.

`08_appendix_figures.R` draws the three figures of Appendix B: the island and district land cover from the aggregated C-CAP masks (with the areas quoted in the thesis's Section 1.2, `results/island_landcover.json`), the terrain of O'ahu and Honolulu from the USGS 10 m DEM (the two subsampled grids in `data/external/`), and the sky view factor diagram for one site, drawn with the same ray casting as `01_site_predictors.R` with the angle of every ray kept.

8 Reproducing everything

```
Rscript analysis/requirements.R # installs the packages (once)
Rscript analysis/run_all.R      # ~30 minutes; writes data/processed/, data/descriptors/ and
results/
```

`results/sessionInfo.txt` records the exact package versions of the thesis run:

Hide

```
cat(readLines(file.path(OUT, "sessionInfo.txt"))[1:12], sep = "\n")
```

```
#> R version 4.3.3 (2024-02-29)
#> Platform: x86_64-pc-linux-gnu (64-bit)
#> Running under: Ubuntu 24.04.4 LTS
#>
#> Matrix products: default
#> BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
#> LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
#>
#> locale:
#> [1] LC_CTYPE=C.UTF-8      LC_NUMERIC=C          LC_TIME=C.UTF-8
#> [4] LC_COLLATE=C.UTF-8    LC_MONETARY=C.UTF-8   LC_MESSAGES=C.UTF-8
#> [7] LC_PAPER=C.UTF-8      LC_NAME=C             LC_ADDRESS=C
```

The three large inputs (the C-CAP tiles, the FEMA footprints and the GSHHG shoreline) are not in the repository; the README says where to download them and where to put them. Without them, `00_prepare_inputs.R` and every model, table and figure that reads the stored descriptors still run.

9 Verification: an independent replication in Python

To guard against an implementation error in any one package, the pipeline up to the results file was written a second time in Python (pandas for the record, rasterio/shapely for the descriptors, statsmodels MixedLM for the models; `replication/python/`) and run on the same inputs. `04_replication_check.R` compares the two results files quantity by quantity and writes `replication/replication_check.csv`:

Hide

```
rp <- read_csv(file.path(root, "replication", "replication_check.csv"), show_col_types = FALSE)
cat(sprintf("%d quantities compared; %d differ by more than 0.01\n", sum(!is.na(rp$Python)),
sum(abs(rp$difference) > 0.01, na.rm = TRUE)))
```

```
#> 117 quantities compared; 3 differ by more than 0.01
```

Hide

```
rp %>% filter(abs(difference) > 0.01) %>% mutate(across(where(is.numeric), ~ signif(.x, 4))) %>%
tab(caption = "The only differences larger than 0.01")
```

The only differences larger than 0.01

quantity	R	Python	unit	difference
Scale rule: max Δ AIC at 50 m	12.34	12.32	NA	0.02036
Interactions: joint LRT chi2	30.43	30.42	NA	0.01037
Night effect: df (lme4 drops the collinear night dummy)	9.00	10.00	NA	-1.00000

The sensor-night ΔT values agree to machine precision; the raster fractions differ by at most 0.2 percentage points at 50 m (the two raster libraries make different choices about pixels on the circle's edge); every coefficient and variance component agrees to three decimals and every AIC and R^2 to two. The residual differences are documented implementation choices: `lme4` drops the night dummy that is collinear with district (so the night-effect test has 9 rather than 10 df), and the two joint-test statistics differ in the second decimal because of optimizer tolerances; the leave-one-site-out folds of the over-parameterized 'Ewa full model, whose fits are the least well determined, are the quantities most sensitive to the optimizer ($r = 0.649$ in R against 0.658 in statsmodels). In other words: the conclusions of the thesis do not depend on which software fitted the models.

10 References for the software

- Bates D, Mächler M, Bolker B, Walker S (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software* 67(1), 1–48. doi:10.18637/jss.v067.i01 (doi:10.18637/jss.v067.i01)
- Fox J, Weisberg S (2019). *An R Companion to Applied Regression*, 3rd ed. Sage.
- Grolemund G, Wickham H (2011). Dates and times made easy with lubridate. *Journal of Statistical Software* 40(3), 1–25. doi:10.18637/jss.v040.i03 (doi:10.18637/jss.v040.i03)
- Hijmans RJ (2024). terra: Spatial data analysis. R package. <https://CRAN.R-project.org/package=terra> (<https://CRAN.R-project.org/package=terra>)
- Johnson GT, Watson ID (1984). The determination of view-factors in urban canyons. *Journal of Climate and Applied Meteorology* 23(2), 329–335. doi:10.1175/1520-0450(1984)023 (doi:10.1175/1520-0450(1984)023<0329:TDOVFI>2.0.CO;2)
- Kuznetsova A, Brockhoff PB, Christensen RHB (2017). lmerTest package: tests in linear mixed effects models. *Journal of Statistical Software* 82(13), 1–26. doi:10.18637/jss.v082.i13 (doi:10.18637/jss.v082.i13)
- Lüdtke D, Ben-Shachar MS, Patil I, Waggoner P, Makowski D (2021). performance: an R package for assessment, comparison and testing of statistical models. *Journal of Open Source Software* 6(60), 3139. doi:10.21105/joss.03139 (doi:10.21105/joss.03139)
- Nakagawa S, Schielzeth H (2013). A general and simple method for obtaining R^2 from generalized linear mixed-effects models. *Methods in Ecology and Evolution* 4(2), 133–142. doi:10.1111/j.2041-210x.2012.00261.x (doi:10.1111/j.2041-210x.2012.00261.x)
- Ooms J (2014). The jsonlite package: a practical and consistent mapping between JSON data and R objects. arXiv:1403.2805
- Pebesma E (2018). Simple features for R: standardized support for spatial vector data. *The R Journal* 10(1), 439–446. doi:10.32614/RJ-2018-009 (doi:10.32614/RJ-2018-009)
- Pedersen TL (2024). patchwork: the composer of plots. R package. <https://CRAN.R-project.org/package=patchwork> (<https://CRAN.R-project.org/package=patchwork>)
- R Core Team (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing, Vienna. <https://www.R-project.org/> (<https://www.R-project.org/>)
- Wickham H (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer. doi:10.1007/978-3-319-24277-4 (doi:10.1007/978-3-319-24277-4)
- Wickham H, Averick M, Bryan J, et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software* 4(43), 1686. doi:10.21105/joss.01686 (doi:10.21105/joss.01686)
- Xie Y, Allaire JJ, Grolemund G (2018). *R Markdown: The Definitive Guide*. Chapman and Hall/CRC.
- Zuur AF, Ieno EN, Walker NJ, Saveliev AA, Smith GM (2009). *Mixed Effects Models and Extensions in Ecology with R*. Springer. doi:10.1007/978-0-387-87458-6 (doi:10.1007/978-0-387-87458-6)